

Feature Specification Document

Project: Crypto Volatility Detection System

Date: November 8, 2025

Version: 1.0

Author: Asli Gulcur

1. Overview

This document specifies the feature engineering and labeling strategy for predicting Bitcoin volatility spikes in real-time. The goal is to build a binary classification model that predicts whether a volatility spike will occur in the next 60 seconds.

2. Prediction Target

2.1 Target Horizon

Horizon: 60 seconds (forward-looking)

Justification:

- **Trading Relevance:** 60 seconds provides actionable time for traders to respond
 - Long enough to execute trades (place orders, adjust positions)
 - Short enough to capture immediate market dynamics
- **Technical Feasibility:** Our feature engineering captures patterns at multiple timescales (10s, 30s, 60s, 120s), with 60s being the central timescale
 - Sufficient historical data to calculate stable features
 - Aligns with rolling window calculations
- **Market Microstructure:** Crypto markets exhibit volatility clustering at minute-level timescales
 - High-frequency trading bots react within seconds
 - News impact propagates within 1-2 minutes
 - 60s captures the complete reaction cycle
- **Literature Support:** Financial research shows 1-minute volatility forecasts are both achievable and useful
 - Standard interval in high-frequency trading research
 - Used by industry for risk management and execution algorithms

Alternative Horizons Considered:

- 30 seconds: Too short, insufficient lead time for human traders
- 120 seconds: Too long, market conditions change significantly

- **60 seconds: Optimal balance** 

2.2 Volatility Proxy

Definition: Rolling standard deviation of midprice returns over the next 60 seconds

Mathematical Formula:

Given:

- Current time: t
- Target horizon: $h = 60$ seconds
- Future time window: $[t, t+h]$
- Midprice returns: r_i for $i \in [t, t+h]$

Volatility Proxy (σ_{future}):

$\sigma_{\text{future}} = \text{StdDev}(r_1, r_2, \dots, r_n)$

Where:

$r_i = (\text{price}_i - \text{price}_{\{i-1\}}) / \text{price}_{\{i-1\}}$

$\text{StdDev} = \sqrt{\sum (r_i - \mu)^2 / (n-1)}$

μ = mean return over window

n = number of returns in window

Implementation:

```
def calculate_future_volatility(df, timestamp, horizon=60):
    """
    Calculate forward-looking volatility over next 'horizon' seconds.

    Args:
        df: DataFrame with 'timestamp' and 'price' columns
        timestamp: Current time point
        horizon: Forward-looking window (seconds)

    Returns:
        float: Standard deviation of returns over next 'horizon' seconds
    """
    # Get future window
    future_df = df[
        (df['timestamp'] > timestamp) &
        (df['timestamp'] <= timestamp + timedelta(seconds=horizon))
    ]

    if len(future_df) < 2:
        return 0.0 # Insufficient future data

    # Calculate returns
    returns = future_df['price'].pct_change().dropna()
```

```
# Return standard deviation (volatility)
return returns.std()
```

Why Standard Deviation of Returns?

- **Industry Standard:** Standard deviation is the universal measure of volatility in finance
- **Scale Invariant:** Returns (%) normalize across different price levels
- **Captures Uncertainty:** Std dev quantifies dispersion = market unpredictability
- **ML-Friendly:** Continuous value, well-behaved distribution

2.3 Label Definition

Binary Classification Target:

```
Label = {
  1  if  $\sigma_{\text{future}} \geq \tau$   (SPIKE - High volatility predicted)
  0  if  $\sigma_{\text{future}} < \tau$     (NORMAL - Low volatility predicted)
}
```

Where:

```
 $\sigma_{\text{future}}$  = Volatility proxy (future 60s volatility)
 $\tau$  = Threshold (spike cutoff)
```

Label Semantics:

- **Label = 1 (Positive Class): "VOLATILITY SPIKE INCOMING"**
 - Action: Increase caution, widen stop-losses, reduce position size
 - Trading: Opportunity for volatility-based strategies (straddles, strangles)
 - Risk: Prepare for rapid price movements
- **Label = 0 (Negative Class): "NORMAL VOLATILITY EXPECTED"**
 - Action: Standard risk management
 - Trading: Normal execution strategies
 - Risk: Lower probability of adverse price movements

3. Threshold Selection (τ)

3.1 Chosen Threshold

$\tau = 0.041799$ (95th percentile of historical volatility_60s distribution)

3.2 Data-Driven Justification

Percentile Analysis Results:

Percentile	Threshold (τ)	Spike Samples	Spike Rate	Assessment
90th	0.015680	3,224	10.0%	⚠ Too sensitive
95th	0.041799	1,612	5.0%	✅ RECOMMENDED
99th	0.075823	323	1.0%	❌ Misses spikes

Selection Rationale:

1. Statistical Balance (5% Spike Rate)

- **Goldilocks Principle:** Not too high, not too low
 - 90th percentile (10%): Too many false positives, alert fatigue
 - 95th percentile (5%): Balanced, manageable alert rate **✅**
 - 99th percentile (1%): Misses legitimate spikes
- **Class Distribution:** 5% positive / 95% negative
 - Sufficient positive samples (1,612) for ML training
 - Not so imbalanced as to require extreme resampling
 - Reflects real-world rarity of significant volatility events

2. Financial Industry Standard

- **95% Confidence Level:** Standard in quantitative finance
 - Value at Risk (VaR) typically uses 95% or 99% confidence
 - Risk management frameworks use 95th percentile for stress testing
 - Regulatory reporting (Basel III) uses similar thresholds
- **Practical Precedent:** Trading systems commonly trigger alerts at 95th percentile
 - Not every small fluctuation (90th would trigger too often)
 - Catches genuinely unusual events (99th would miss actionable spikes)

3. Captures Real Market Events

- **Observed Spike Event:** Our dataset includes a volatility spike at 22:08:30
 - Maximum volatility: 0.3847 (384.7 basis points)
 - Well above 95th percentile threshold (0.0418)
 - Threshold successfully identifies this event **✅**
- **Event Characteristics:**
 - Price drop: \$103,719 → \$103,434 (-0.27%)
 - Volatility surge: 0.0020 → 0.0096 (+380%)
 - Duration: ~8 seconds of extreme movement
 - **95th percentile threshold captures this entire event**

4. Actionable for Trading

- **Alert Frequency:** ~5% of time = ~3 alerts per hour (at 1 tick/second rate)
 - Manageable for human monitoring
 - Not overwhelming for automated systems
 - Provides useful signal without noise
- **Lead Time:** 60-second horizon provides reaction time
 - Cancel open orders before adverse execution
 - Adjust hedges and risk exposure
 - Enter volatility arbitrage positions

5. Model Training Considerations

- **Sample Size:** 1,612 positive samples (spikes) in our dataset
 - Sufficient for training robust ML models
 - Can use stratified cross-validation
 - No need for synthetic oversampling (yet)
- **Evaluation Metrics:** 5% spike rate enables meaningful metrics
 - Precision: Proportion of predicted spikes that are real
 - Recall: Proportion of real spikes caught
 - F1-score: Balanced harmonic mean
 - ROC-AUC: Discriminative ability across thresholds

6. Visualization Evidence

Percentile Plot: Shows exponential increase in volatility at tail

- 90th: 0.0157 (gradual increase)
- 95th: 0.0418 (sharp jump) ← Clear inflection point
- 99th: 0.0758 (extreme outliers only)

95th percentile sits at the "elbow" where normal volatility transitions to spike territory.

Distribution Plot: Histogram shows:

- Most data concentrated below 0.01 (calm market)
- 95th percentile line cleanly separates "normal" from "spike" regions
- Visual validation of threshold placement

3.3 Alternative Thresholds Rejected

90th Percentile ($\tau = 0.015680$) - TOO SENSITIVE

- **Why rejected:** 10% spike rate too high

- 3,224 samples flagged as spikes (excessive)
- Alert fatigue: Too many false alarms
- Reduces trust in system
- **When to use:** Extremely risk-averse applications
 - Example: High-leverage trading (100x leverage)
 - Need very early warning, willing to tolerate false positives

99th Percentile ($\tau = 0.075823$) - TOO CONSERVATIVE

- **Why rejected:** Only 1% spike rate
 - 323 samples = only the most extreme events
 - Misses moderate but significant volatility spikes
 - Reduces model utility
- **When to use:** Low-frequency, high-impact alerts
 - Example: Risk committee notifications
 - Only want to be alerted to market crises

3.4 Sensitivity Analysis

Threshold Impact on Model Metrics:

Assuming baseline model accuracy of 90% on normal volatility:

Threshold	Spike Rate	Expected Precision	Expected Recall	Trade-off
90th (0.0157)	10%	~50% (many FPs)	~95% (catches most)	High recall, low precision
95th (0.0418)	5%	~70%	~85%	Balanced 
99th (0.0758)	1%	~85% (few FPs)	~60% (misses many)	High precision, low recall

Our Choice: Prioritize balanced performance (F1-score) over extreme precision or recall.

4. Feature Set

4.1 Complete Feature List

The ML model will use the following **13 features** calculated from real-time Bitcoin tick data:

Price Features (3)

1. **price** - Current Bitcoin midprice (USD)
2. **bid** - Best bid price (highest buy order)
3. **ask** - Best ask price (lowest sell order)

Return Features (3) - Momentum & Direction

4. **return_10s** - % change over last 10 seconds
5. **return_30s** - % change over last 30 seconds
6. **return_60s** - % change over last 60 seconds

Rationale: Multi-scale returns capture:

- Short-term momentum (10s): Recent price direction
- Medium-term trend (30s): Sustained movement
- Long-term direction (60s): Overall market sentiment

Volatility Features (3) - Our Key Predictors

7. **volatility_30s** - Std dev of returns over last 30 seconds
8. **volatility_60s** - Std dev of returns over last 60 seconds
9. **volatility_120s** - Std dev of returns over last 120 seconds

Rationale: Multi-scale volatility captures:

- Recent instability (30s): Current market jitters
- Medium-term risk (60s): Sustained volatility level
- Long-term context (120s): Overall market regime

Hypothesis: When short-term volatility diverges from long-term volatility, a spike is likely.

Spread Features (1) - Market Liquidity

10. **spread** - $(\text{ask} - \text{bid}) / \text{midprice}$ (relative bid-ask spread)

Rationale:

- Widening spread → Market uncertainty, liquidity drying up
- Often precedes volatility spikes (traders pulling orders)
- Early warning indicator

Intensity Features (3) - Trading Activity

11. **intensity_30s** - Trades per second over last 30 seconds
12. **intensity_60s** - Trades per second over last 60 seconds
13. **intensity_120s** - Trades per second over last 120 seconds

Rationale:

- Sudden activity surge → News event or large order
- Often precedes price movement and volatility
- Captures market participation level

4.2 Feature Engineering Details

Calculation Method: Rolling Windows

All features use **rolling time windows** (not tick windows):

- Window = Last X seconds of data
- Recalculated on every new tick
- Memory-efficient using circular buffer (deque with maxlen=500)

Example:

```
# At time T = 10:00:00
return_30s = (price_T - price_{T-30s}) / price_{T-30s}

# At time T = 10:00:01 (new tick arrives)
return_30s = (price_T - price_{T-30s}) / price_{T-30s}
# Window slides forward automatically
```

Edge Case Handling

- **Insufficient data:** Return 0.0 if window not yet filled
- **Division by zero:** Protected with `if price > 0` checks
- **Missing ticks:** Time-based windows handle irregular tick arrival

4.3 Feature Importance (Expected)

Based on domain knowledge and EDA:

Feature	Expected Importance	Reasoning
volatility_30s	★★★★★	Recent volatility predicts future volatility (autocorrelation)
volatility_60s	★★★★★	Direct predictor of target (σ_{future} at 60s horizon)
volatility_120s	★★★★	Long-term context, divergence from short-term signals regime change
return_10s	★★★	Sudden returns often precede volatility
spread	★★★	Liquidity indicator, widening spread = impending volatility
intensity_30s	★★★	Activity surge signals market event
return_30s/60s	★★	Additional momentum context
intensity_60s/120s	★★	Contextual activity levels
price/bid/ask	★	Absolute values less predictive than derived features

Note: Actual importance will be measured via:

- Permutation importance (Milestone 3)
 - SHAP values (Milestone 3)
 - Feature ablation studies (Milestone 3)
-

5. Data Quality & Validation

5.1 Dataset Statistics

Source: Real Bitcoin (BTC-USD) tick data from Coinbase WebSocket

Collection Date: November 7, 2025

Duration: 57.6 seconds

Total Records: 32,233 feature vectors

Quality Metrics:

-  Missing values: 0 (0%)
-  Duplicate timestamps: 0 (after deduplication fix)
-  Outliers: Present in tail (volatility spike event - legitimate, not error)
-  Data completeness: 100% (all 13 features present)

5.2 Validation Strategy

Temporal Split (Time-Series Aware):

- **Training:** First 70% chronologically (0 to 40 seconds)
- **Validation:** Next 15% (40 to 49 seconds)
- **Test:** Last 15% (49 to 58 seconds)

Why temporal split?

- Prevents data leakage (no future information in past)
- Simulates real production scenario (train on history, predict future)
- Standard practice for time-series ML

Stratification:

- Ensure spike events (label=1) present in all splits
 - May need oversampling if spike concentration in one period
-

6. Reproducibility

6.1 Feature Calculation Determinism

Requirement: Replay must produce identical features to live ingestion

Implementation:

- All calculations use deterministic functions (no randomness)

- Timestamp-based windows (not tick-count based)
- Same buffer management logic (deque with maxlen=500)
- Identical floating-point arithmetic

Validation:

```
# Run live featurizer
python features/featurizer.py # Produces features_live.parquet

# Run replay
python scripts/replay.py      # Produces features_replay.parquet

# Compare (should be identical)
diff <(parquet-tools head features_live.parquet) \
    <(parquet-tools head features_replay.parquet)
```

6.2 Versioning

Feature Set Version: 1.0**Changes from v0.x:**

- N/A (initial version)

Future changes will increment version:

- v1.1: Minor (add features, keep existing)
- v2.0: Major (remove features, change calculations)

7. Production Considerations

7.1 Latency Requirements

Target: < 10ms per feature calculation

- **Current:** ~2ms measured (well within target) 
- **Bottlenecks:** Standard deviation calculation (requires iteration)
- **Optimization:** Pre-computed rolling statistics if needed

7.2 Monitoring

Feature Drift Detection (Evidently):

- Compare feature distributions weekly
- Alert if volatility baseline shifts > 2x
- Retrain model if drift persists > 1 week

Data Quality Checks:

- Missing value rate < 0.1% (alert if exceeded)

- Duplicate rate < 0.1% (alert if exceeded)
- Feature range validation (e.g., spread > 0, volatility >= 0)

7.3 Model Retraining Triggers

Automatic Retraining If:

1. Feature drift detected (distribution shift > 2σ)
2. Model accuracy drops < 80% on validation set
3. Scheduled weekly retraining (capture latest market patterns)

8. Summary

Key Decisions

Parameter	Value	Justification
Target Horizon	60 seconds	Actionable for traders, aligns with feature timescales
Volatility Proxy	Std dev of returns	Industry standard, scale-invariant
Threshold (τ)	0.041799 (95th percentile)	Balances sensitivity (5% spike rate), captures real events
Label Definition	Binary (spike vs normal)	Simplifies problem, actionable predictions
Feature Count	13 features	Multi-scale returns, volatility, spread, intensity

Next Steps (Milestone 3)

1.  Feature specification complete
2.  Train baseline ML model (logistic regression, random forest)
3.  Evaluate with precision, recall, F1-score, ROC-AUC
4.  Feature importance analysis (SHAP values)
5.  Hyperparameter tuning
6.  Production deployment with monitoring

Appendix A: Threshold Selection Plots

See: [notebooks/eda.ipynb](#) Section 6: "Threshold Selection for Spike Detection"

Plots Included:

1. Volatility distribution with threshold lines (90th, 95th, 99th percentiles)
2. Percentile vs threshold value curve (shows exponential tail)
3. Threshold impact analysis table

Appendix B: Feature Calculation Code

See: `features/featurizer.py`

Key Methods:

- `calculate_return(window_seconds)` - Lines 250-280
 - `calculate_volatility(window_seconds)` - Lines 282-320
 - `calculate_spread()` - Lines 322-335
 - `calculate_intensity(window_seconds)` - Lines 337-365
 - `calculate_features()` - Lines 367-410 (main feature generation)
-

Document Status:  Complete

Review Date: November 8, 2025

Approved By: Asli Gulcur